

Program 7: Maven Project (all 3 combined in Firefox) To Connect To Web Servers Using Selenium Dependency

Step 1: Install geckodriver (Firefox webdriver)

wget

<https://github.com/mozilla/geckodriver/releases/download/v0.36.0/geckodriver-v0.36.0-linux64.tar.gz>

Step 2: Download GeckoDriver:

```
tar -xvzf geckodriver-v0.36.0-linux64.tar.gz
```

Step 3: Extract downloaded folder:

```
sudo mv geckodriver /usr/local/bin/
```

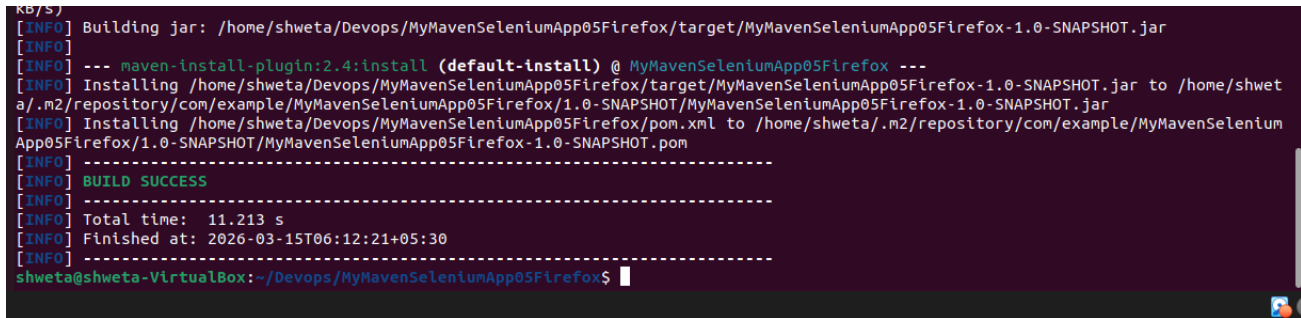
Step 4: Move driver to system path

```
sudo chmod +x /usr/local/bin/geckodriver
```

```
shweta@shweta-VirtualBox:~$ wget https://github.com/mozilla/geckodriver/releases/download/v0.36.0/geckodriver-v0.36.0-linux64.tar.gz
--2026-03-15 06:22:39-- https://github.com/mozilla/geckodriver/releases/download/v0.36.0/geckodriver-v0.36.0-linux64.tar.gz
Resolving github.com (github.com)... 20.207.73.82
Connecting to github.com (github.com)|20.207.73.82|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://release-assets.githubusercontent.com/github-production-release-asset/25354393/73faebef-d9f0-4bca-a87c-a8332a5f9deb?sp=r&sv=2018-11-09&spr=b&spr=https&se=2026-03-15T01%3A35%3A5Z2&rscd=attachment%3B+filename%3Dgeckodriver-v0.36.0-linux64.tar.gz&rscst=application%2Foctet-stream&skoid=96c2d410-5711-43a1-aedd-ab1947aa7ab0&sktid=398a6654-997b-47e9-b12b-9515b896b4de&skt=2026-03-15T00%3A35%3A00Z&ske=2026-03-15T01%3A35%3A5Z2&skvs=b&skv=v0.36.0-linux64.tar.gz&sig=lml1tibx2Fx8dvaxul4SmPQg%2B9hmJ5zcQBbd8ptQTjUj0%3D&jwt=eYJ0eXAIOIJKVlQICLChhgciOIJiuZIINlj9_eYJCpCMIoiJnaXRodmWYu29TiIwiYXVkIjoicmVsZWZkZFZZShc3NldHMuzZl0ahVidXLncmNvbmlnbnQuY29TiIwia2VSijoaia2VSMSIsImV4CC1MTczMzUZnlJMswibmJMiJoXNzczNTMOTQKLClwYXRoIjoicmVsZWZkZFZZSVh3ZGVHJVjdGlbvblbGNvcmlud2luZG93cySuzXqifQ.Khti-YeuNSTFGPg6UIzmZYVPWRDL6Z29PiVLHJJ20yg&response-content-disposition=attachment%3B%20filename%3Dgeckodriver-v0.36.0-linux64.tar.gz&response-content-type=application%2Foctet-stream [following]
--2026-03-15 06:22:40-- https://release-assets.githubusercontent.com/github-production-release-asset/25354393/73faebef-d9f0-4bca-a87c-a8332a5f9deb?sp=r&sv=2018-11-09&spr=b&spr=https&se=2026-03-15T01%3A35%3A5Z2&rscd=attachment%3B+filename%3Dgeckodriver-v0.36.0-linux64.tar.gz&rscst=application%2Foctet-stream&skoid=96c2d410-5711-43a1-aedd-ab1947aa7ab0&sktid=398a6654-997b-47e9-b12b-9515b896b4de&skt=2026-03-15T00%3A35%3A00Z&ske=2026-03-15T01%3A35%3A5Z2&skvs=b&skv=v0.36.0-linux64.tar.gz&sig=lml1tibx2Fx8dvaxul4SmPQg%2B9hmJ5zcQBbd8ptQTjUj0%3D&jwt=eYJ0eXAIOIJKVlQICLChhgciOIJiuZIINlj9_eYJCpCMIoiJnaXRodmWYu29TiIwiYXVkIjoicmVsZWZkZFZZShc3NldHMuzZl0ahVidXLncmNvbmlnbnQuY29TiIwia2VSijoaia2VSMSIsImV4CC1MTczMzUZnlJMswibmJMiJoXNzczNTMOTQKLClwYXRoIjoicmVsZWZkZFZZSVh3ZGVHJVjdGlbvblbGNvcmlud2luZG93cySuzXqifQ.Khti-YeuNSTFGPg6UIzmZYVPWRDL6Z29PiVLHJJ20yg&response-content-disposition=attachment%3B%20filename%3Dgeckodriver-v0.36.0-linux64.tar.gz&response-content-type=application%2Foctet-stream
Resolving release-assets.githubusercontent.com (release-assets.githubusercontent.com)... 185.199.108.133, 185.199.110.133, 185.199.111.133, ...
Connecting to release-assets.githubusercontent.com (release-assets.githubusercontent.com)[185.199.108.133]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 2306404 (2.2M) [application/octet-stream]
Saving to: ‘geckodriver-v0.36.0-linux64.tar.gz’
```

Step 5: create a maven project on terminal

```
mvn archetype:generate -DgroupId=com.example -DartifactId=
MyMavenSeleniumApp05Firefox -DarchetypeArtifactId=maven-
archetype-quickstart -DinteractiveMode=false
```



```
KB/S)
[INFO] Building jar: /home/shweta/Devops/MyMavenSeleniumApp05Firefox/target/MyMavenSeleniumApp05Firefox-1.0-SNAPSHOT.jar
[INFO] --- maven-install-plugin:2.4:install (default-install) @ MyMavenSeleniumApp05Firefox ---
[INFO] Installing /home/shweta/Devops/MyMavenSeleniumApp05Firefox/target/MyMavenSeleniumApp05Firefox-1.0-SNAPSHOT.jar to /home/shweta/.m2/repository/com/example/MyMavenSeleniumApp05Firefox/1.0-SNAPSHOT/MyMavenSeleniumApp05Firefox-1.0-SNAPSHOT.jar
[INFO] Installing /home/shweta/Devops/MyMavenSeleniumApp05Firefox/pom.xml to /home/shweta/.m2/repository/com/example/MyMavenSeleniumApp05Firefox/1.0-SNAPSHOT/MyMavenSeleniumApp05Firefox-1.0-SNAPSHOT.pom
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 11.213 s
[INFO] Finished at: 2026-03-15T06:12:21+05:30
[INFO] -----
shweta@shweta-VirtualBox: ~/Devops/MyMavenSeleniumApp05Firefox$
```

Step 2: Edit the pom.xml file.

Add the Selenium dependency by going to the Maven Central Repository, searching for Selenium, selecting **Selenium Java**, choosing the latest version, and copying the dependency code. Paste this dependency into the pom.xml file.

```
<project
xmlns="http://maven.apache.org/POM/4.0.0
"

xmlns:xsi="http://www.w3.org/2001/XMLSchema
schema-instance"

xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId>
```

```
<artifactId>MyMavenSeleniumApp05Firefox</artifactId>

<packaging>jar</packaging>

<version>1.0-SNAPSHOT</version>

<name>MyMavenSeleniumApp05Firefox</name>

<url>http://maven.apache.org</url>

<dependencies>

  <dependency>

    <groupId>junit</groupId>

    <artifactId>junit</artifactId>

    <version>3.8.1</version>

    <scope>test</scope>

  </dependency>

<!-- https://mvnrepository.com/artifact/org.seleniumhq.selenium/selenium-java -->

<dependency>

  <groupId>org.seleniumhq.selenium</groupId>

  <artifactId>selenium-java</artifactId>

  <version>4.29.0</version>

</dependency>

</dependencies>

<!-- Build: Configuring plugins and build settings -->
```

```
<build>

<plugins>

<!-- Example: Maven Compiler Plugin to compile Java code -->

<plugin>

<groupId>org.apache.maven.plugins</groupId>

<artifactId>maven-compiler-plugin</artifactId>

<version>3.8.1</version>

<configuration>

<source>1.7</source>

<target>1.7</target>

</configuration>

</plugin>

<!-- Example: Maven Surefire Plugin to run tests -->

<plugin>

<groupId>org.apache.maven.plugins</groupId>

<artifactId>maven-surefire-plugin</artifactId>

<version>2.22.2</version>

</plugin>

<plugin>

    <groupId>org.apache.maven.plugins</groupId>
```

```
<artifactId>maven-jar-plugin</artifactId>

<version>3.0.1</version>

<configuration>

    <archive>

<manifestEntries>

    <Main-Class>com.example.App</Main-Class>

</manifestEntries>

</archive>

</configuration>

</plugin>

</plugins>

</build>

</project>
```

3: Before writing the App.java file, inspect the Practice Test Automation webpage to find the element IDs required for automation.

```
package com.example;
```

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
```

```
public class App {
```

```
public static void main(String[] args) {

    WebDriver driver = new ChromeDriver();

    // ----- 1. SauceDemo Login -----
    driver.get("https://www.saucedemo.com/");
    driver.manage().window().maximize();

    driver.findElement(By.id("user-name")).sendKeys("standard_user");
    driver.findElement(By.id("password")).sendKeys("secret_sauce");
    driver.findElement(By.id("login-button")).click();

    // ----- 2. Practice Test Login -----
    driver.get("https://practicetestautomation.com/practice-test-login/");
    driver.manage().window().maximize();

    driver.findElement(By.id("username")).sendKeys("student");
    driver.findElement(By.id("password")).sendKeys("Password123");
    driver.findElement(By.id("submit")).click();

    // ----- 3. Automation Exercise Search -----
    driver.get("https://automationexercise.com/products");
    driver.manage().window().maximize();

    driver.findElement(By.id("search_product")).sendKeys("Blue Top");
    driver.findElement(By.id("submit_search")).click();

    // driver.quit(); // optional
}
}
```

Step 4: build and run the maven application**mvn clean install**

```
ko7s/
[INFO] Building jar: /home/shweta/Devops/MyMavenSeleniumApp05Firefox/target/MyMavenSeleniumApp05Firefox-1.0-SNAPSHOT.jar
[INFO] --- maven-install-plugin:2.4:install (default-install) @ MyMavenSeleniumApp05Firefox ---
[INFO] Installing /home/shweta/Devops/MyMavenSeleniumApp05Firefox/target/MyMavenSeleniumApp05Firefox-1.0-SNAPSHOT.jar to /home/shweta/.m2/repository/com/example/MyMavenSeleniumApp05Firefox/1.0-SNAPSHOT/MyMavenSeleniumApp05Firefox-1.0-SNAPSHOT.jar
[INFO] Installing /home/shweta/Devops/MyMavenSeleniumApp05Firefox/pom.xml to /home/shweta/.m2/repository/com/example/MyMavenSeleniumApp05Firefox/1.0-SNAPSHOT/MyMavenSeleniumApp05Firefox-1.0-SNAPSHOT.pom
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 11.213 s
[INFO] Finished at: 2026-03-15T06:12:21+05:30
[INFO] -----
shweta@shweta-VirtualBox:~/Devops/MyMavenSeleniumApp05Firefox$
```

Step 5: run the below command**mvn exec:java -Dexec.mainClass="com.example.App"**

```
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 26.846 s
[INFO] Finished at: 2026-03-17T05:03:02Z
[INFO] -----
```

Swag Labs

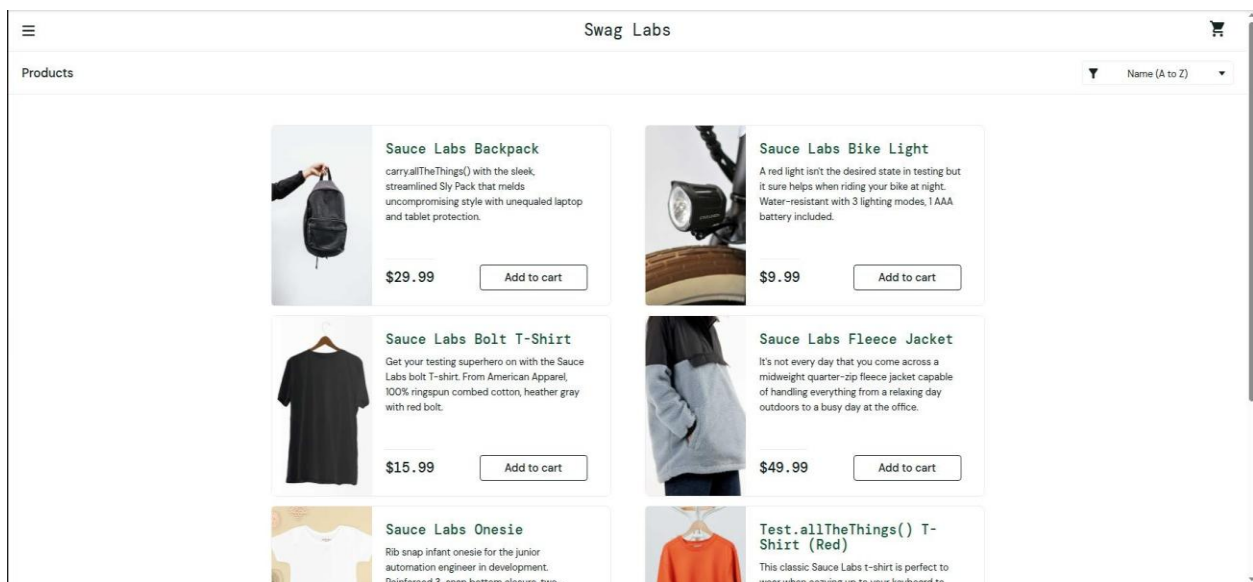
Login

Accepted usernames are:

standard_user
locked_out_user
problem_user
performance_glitch_user
error_user
visual_user

Password for all users:

secret_sauce





HOME PRACTICE COURSES BLOG CONTACT

Test login

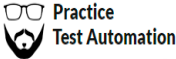
This is a simple Login page. Students can use this page to practice writing simple positive and negative Login tests. Login functionality is something that most of the test automation engineers need to automate.

Use next credentials to execute Login:
Username: **student**
Password: **Password123**

Username

Password

Submit

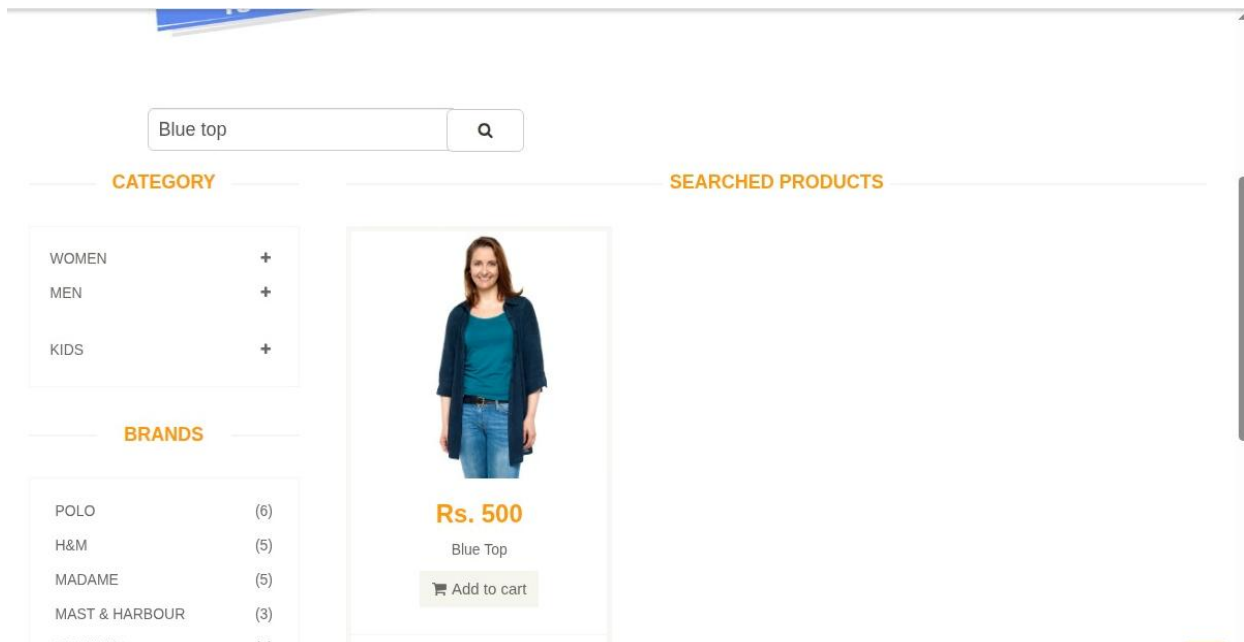


HOME PRACTICE COURSES BLOG CONTACT

Logged In Successfully

Congratulations student. You successfully logged in!

Log out



Pushing MyMavenSeleniumApp05Firefox to Github:

```
shweta@shweta-VirtualBox: ~/Devops/MyMavenSeleniumApp05Firefox
shweta@shweta-VirtualBox:~$ cd Devops
shweta@shweta-VirtualBox:~/Devops$ cd MyMavenSeleniumApp05Firefox
shweta@shweta-VirtualBox:~/Devops/MyMavenSeleniumApp05Firefox$ git init
Reinitialized existing Git repository in /home/shweta/Devops/MyMavenSeleniumApp05Firefox/.git/
shweta@shweta-VirtualBox:~/Devops/MyMavenSeleniumApp05Firefox$ git add .
shweta@shweta-VirtualBox:~/Devops/MyMavenSeleniumApp05Firefox$ git commit -m "first commit"
On branch master
Your branch is up to date with 'origin/master'.

nothing to commit, working tree clean
shweta@shweta-VirtualBox:~/Devops/MyMavenSeleniumApp05Firefox$ git remote add origin https://github.com/Shweta311204/MyMavenSeleniumApp05Firefox.git
error: remote origin already exists.
shweta@shweta-VirtualBox:~/Devops/MyMavenSeleniumApp05Firefox$ git remote set-url origin https://ghp_iQyXf5...IO4V0ZLR8oBxoMcWt74eids0nM4p3@github.com/Shweta311204/MyMavenSeleniumApp05Firefox.git
shweta@shweta-VirtualBox:~/Devops/MyMavenSeleniumApp05Firefox$ git push -u origin master
Branch 'master' set up to track remote branch 'master' from 'origin'.
Everything up-to-date
shweta@shweta-VirtualBox:~/Devops/MyMavenSeleniumApp05Firefox$
```

Setting Up Jenkins Pipeline for MyMavenSeleniumApp05Firefox

Step 1: To deploy **MyMavenSeleniumApp05firefox**, open the terminal, cd into the **MyMavenSeleniumApp05Firefox** folder, create a **Jenkinsfile** containing the pipeline code, and then commit and push it to GitHub.

```
pipeline {  
  
    agent any // Use any available agent tools {  
  
        maven 'Maven' // Ensure this matches the name configured in Jenkins  
    }  
  
    stages {  
  
        stage('Checkout') { steps {  
  
            git branch: 'master', url:  
            'https://github.com/Shweta311204/MyMavenSeleniumApp05Firefox.git'  
        }  
    }  
  
        stage('Build') { steps {  
  
            sh 'mvn clean package' // Run Maven build  
        }  
    }  
  
        stage('Test') { steps {  
  
            sh 'mvn test' // Run unit tests  
        }  
    }  
  
        stage('Run Application') { steps {  
  
            // Start the JAR application
```

```
sh 'mvn exec:java -Dexec.mainClass="com.example.App"'
}
}
}

post {

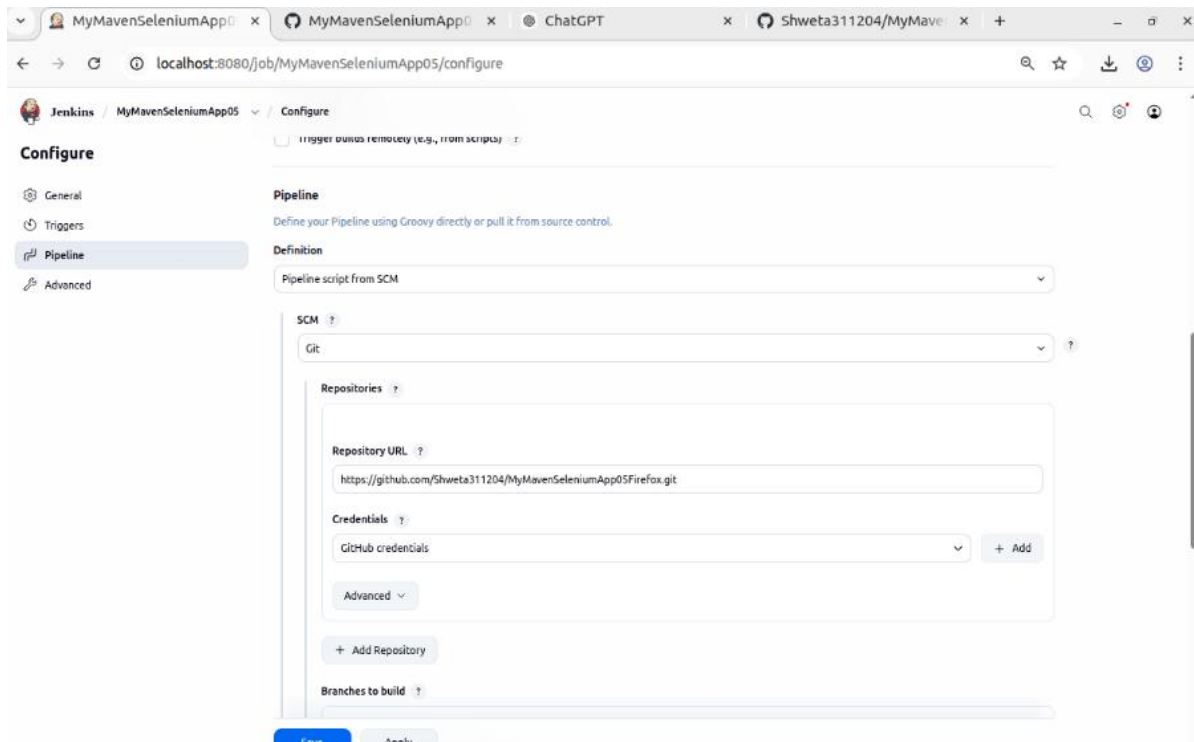
success {

echo 'Build and deployment successful!'
}

failure {

echo 'Build failed!'
}
}
}
```

Step 2: To deploy **MyMavenSeleniumApp05Firefox**, you first need to create a Jenkins pipeline job that will use the Jenkinsfile from your project repository. Then, go to the Jenkins home page and click on “**New Item**” to create a job, enter the item name (e.g., *MyMavenSeleniumApp05*), and select **Pipeline** as the project type. This option allows you to build, test, and deploy applications using a Jenkinsfile. After selecting the pipeline type, click “**OK**” to proceed to the job configuration page. In **MyMavenSeleniumApp05**, go to the **Pipeline** section and under **Definition**, select **Pipeline script from SCM**. Choose **Git** as the SCM, and under **Repository URL**, paste the GitHub URL of **MyMavenSeleniumApp05**. In the **Credentials** field, select the previously created credential (e.g., *GitHub Credential*). Finally, in **Script Path**, enter the name of the Jenkinsfile you created in the *MyMavenSeleniumApp01* folder and pushed to GitHub. Then click **Apply** and **Save** to finalize the configuration.



Step 4: After clicking **Apply and Save**, go back to **MyMavenSeleniumApp02** in **Jenkins** and click **Build Now**. If you get an error like:

java.lang.NoClassDefFoundError:
io/github/bonigarcia/wdm/WebDriverManager

this error occurs when the application is unable to find required external libraries (such as Selenium WebDriver and WebDriverManager) at runtime. Even though the project builds successfully, the dependencies are not included inside the final JAR file, so Jenkins fails while executing the program.

To solve this issue, instead of running the application using a JAR file, we use Maven execution so that all dependencies defined in pom.xml are properly loaded during runtime execution.

This ensures that Selenium and WebDriverManager libraries are available when the program runs in Jenkins.

Replace the existing execution method with the following:

```
mvn exec:java -Dexec.mainClass="com.example.App"
```

Jenkins Pipeline Update:

```
stage('Run Application') { steps {  
  
sh 'mvn exec:java -Dexec.mainClass="com.example.App"'  
}  
}
```

This stage is used to execute the Java application directly through Maven so that all dependencies defined in the project are included during runtime.

pom.xml Dependencies (Required Fix):

```
<dependency>  
  
<groupId>org.seleniumhq.selenium</groupId>  
  
<artifactId>selenium-java</artifactId>  
  
<version>4.29.0</version>  
  
</dependency>  
  
<dependency>  
  
<groupId>io.github.bonigarcia</groupId>  
  
<artifactId>webdrivermanager</artifactId>  
  
<version>5.7.0</version>  
  
</dependency>
```

These dependencies are required to enable Selenium automation and automatic browser driver management during execution.

Changes made in App.java to solve the error:

To resolve the **NoClassDefFoundError (WebDriverManager missing at runtime)**, the following changes were ensured in App.java:

Added **WebDriverManager setup** to automatically handle Firefoxriver dependency: `WebDriverManager.chromedriver().setup();`

This line automatically downloads, sets up, and manages the correct version of `FirefoxDriver` required for the installed `Firefox` browser. It removes the need to manually download or configure the driver path and ensures compatibility between `Firefox` and `FirefoxDriver` during Jenkins execution.

Configured **FirefoxOptions for Jenkins/Linux environment** to avoid browser crash in headless mode:

```
FirefoxOptions options = new FirefoxOptions(); options.addArguments("--headless=new"); options.addArguments("--no-sandbox"); options.addArguments("--disable-dev-shm-usage"); options.addArguments("--disable-gpu"); options.addArguments("--remote-allow-origins=*");
```

These options ensure that `Firefox` runs in a headless (no UI) mode suitable for Jenkins/Linux servers and prevent common issues like browser crash, sandbox errors, and memory limitations during execution.

Used **FirefoxDriver with options** instead of normal driver: `WebDriver driver = new FirefoxDriver(options);`

This ensures that the browser is launched with the configured `FirefoxOptions` settings (such as headless mode and Jenkins-safe arguments), making the automation stable and suitable for execution in CI/CD environments like Jenkins.

The following are the updated pom.xml, App.java, and Jenkinsfile: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        http://maven.apache.org/maven-v4_0_0.xsd">
```

```
<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId>

<artifactId>MyMavenSeleniumApp05Firefox</artifactId>

<version>1.0-SNAPSHOT</version>

<packaging>jar</packaging>

<name>MyMavenSeleniumApp05Firefox</name>

<!-- Java Version -->

<properties>

    <maven.compiler.source>11</maven.compiler.source>

    <maven.compiler.target>11</maven.compiler.target>

</properties>

<dependencies>

    <!-- JUnit (Test only) -->

    <dependency>

        <groupId>junit</groupId>

        <artifactId>junit</artifactId>

        <version>4.13.2</version>

        <scope>test</scope>

    </dependency>

    <!-- Selenium -->
```



```
<dependency>

  <groupId>org.seleniumhq.selenium</groupId>

  <artifactId>selenium-java</artifactId>

  <version>4.41.0</version>

</dependency>

</dependencies>

<build>

  <plugins>

    <!-- Compiler Plugin -->

    <plugin>

      <groupId>org.apache.maven.plugins</groupId>

      <artifactId>maven-compiler-plugin</artifactId>

      <version>3.11.0</version>

    </plugin>

    <!-- Surefire Plugin (for tests) -->

    <plugin>

      <groupId>org.apache.maven.plugins</groupId>

      <artifactId>maven-surefire-plugin</artifactId>

      <version>3.2.5</version>
```

```
</plugin>

<!-- Shade Plugin (creates fat JAR with dependencies) -->

<plugin>

  <groupId>org.apache.maven.plugins</groupId>

  <artifactId>maven-shade-plugin</artifactId>

  <version>3.5.0</version>

  <executions>

    <execution>

      <phase>package</phase>

      <goals>

        <goal>shade</goal>

      </goals>

      <configuration>

        <createDependencyReducedPom>>false</createDependencyReducedPom>

        <transformers>

          <transformer
implementation="org.apache.maven.plugins.shade.resource.ManifestResourceTran
sformer">

            <mainClass>com.example.App</mainClass>

          </transformer>

        </transformers>
```

```
        </configuration>
    </execution>
</executions>
</plugin>
</plugins>
</build>
</project>
```

App.java

```
package com.example;

import org.openqa.selenium.*;
import org.openqa.selenium.firefox.FirefoxDriver;
import org.openqa.selenium.firefox.FirefoxOptions;
import org.openqa.selenium.support.ui.*;
import java.time.Duration;

public class App {

    public static void main(String[] args) {

        // Headless Firefox (MANDATORY for Jenkins)

        FirefoxOptions options = new FirefoxOptions();

        options.addArguments("--headless");
```

```
WebDriver driver = new FirefoxDriver(options);

driver.manage().window().setSize(new Dimension(1920,1080));


WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(15));

// ----- Saucedemo -----

driver.get("https://www.saucedemo.com/");


    wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("user-
name"))).sendKeys("standard_user");

driver.findElement(By.id("password")).sendKeys("secret_sauce");

driver.findElement(By.id("login-button")).click();


System.out.println("SauceDemo login successful");


// ----- New Tab -----

driver.switchTo().newWindow(WindowType.TAB);


// ----- Automation Exercise -----

driver.get("https://automationexercise.com/products");
```

```
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("search_product"))).sendKeys("Men Tshirt");
```

```
driver.findElement(By.id("submit_search")).click();
```

```
WebElement product = wait.until(
```

```
    ExpectedConditions.elementToBeClickable(By.cssSelector("a[data-product-id='2']"))
```

```
);
```

```
((JavascriptExecutor) driver).executeScript("arguments[0].scrollIntoView({block:'center'});", product);
```

```
((JavascriptExecutor) driver).executeScript("arguments[0].click();", product);
```

```
WebElement viewCart = wait.until(
```

```
    ExpectedConditions.elementToBeClickable(By.cssSelector("#cartModal a[href='/view_cart']"))
```

```
);
```

```
((JavascriptExecutor) driver).executeScript("arguments[0].click();", viewCart);
```

```
System.out.println("Automation Exercise product added to cart");
```

```
// ----- New Tab -----
```

```
driver.switchTo().newWindow(WindowType.TAB);
```

```
// ----- Practice Test Automation -----
```

```
driver.get("https://practicetestautomation.com/practice-test-login/");
```

```
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("username"))).sendKeys("student");

    driver.findElement(By.id("password")).sendKeys("Password123");

    driver.findElement(By.id("submit")).click();


    System.out.println("Practice Test Automation login successful");


    driver.quit();
}
}
```

Jenkinsfile:

```
pipeline {
    agent any // Use any available agent

    tools {
        maven 'Maven' // Ensure this matches the name configured in Jenkins
        jdk 'JDK'
    }

    stages {
```

```
stage('Checkout') {  
    steps {  
        git branch: 'master', url:  
'https://github.com/Shweta311204/MyMavenSeleniumApp05Firefox.git'  
    }  
}  
  
stage('Build') {  
    steps {  
        sh 'mvn clean package' // Run Maven build  
    }  
}  
  
stage('Test') {  
    steps {  
        sh 'mvn test' // Run unit tests  
    }  
}  
  
stage('Run Application') {  
    steps {  
        // Start the JAR application  
        sh 'java -jar target/MyMavenSeleniumApp02-1.0-SNAPSHOT.jar'  
    } } }  
  
post {
```

```

success {

    echo 'Build and deployment successful!' }

    failure {

    echo 'Build failed!'

    }}}

```

Step 5: After making the changes, go back to **MyMavenSeleniumApp05Firefox** in Jenkins and click **Build Now**. The build should now complete successfully without any errors.

This happens because the dependencies (such as **Selenium** and **WebDriverManager**) are properly included in the project through Maven configuration, and the application is executed using `mvn exec:java`. So during runtime, Jenkins is able to load all required libraries correctly, and issues like **java.lang.NoClassDefFoundError** are resolved.

